

Hermes Polymarket Copy Trading Bot

The exact viewer-facing build prompt, with the Max HQ add-on phrase removed from the opening line.

Safety first: paper trading only, no private keys, no live execution, no real trades by default.

Hermes Polymarket Copy Trading Bot Build Prompt

Build me a Hermes-powered self-improving Polymarket copy trading bot with a Vercel dashboard.

Important:

- This is not financial advice.
- Version one must not place real trades.
- Version one must paper trade only.
- Do not ask for private keys.
- Do not store private keys.
- Do not sign transactions.
- Do not spend money.
- Do not execute real trades.
- The long-term goal is eventual autonomy, but only after paper trading proves the edge.
- If any API fails, show the real error and stop. Do not fake live data.
- Demo or seed data is allowed only if clearly labeled as demo data.
- Use environment variables for optional API keys.
- Redact secrets in logs and UI.

Goal:

Create a working local app and Vercel-ready dashboard that lets Hermes Agent operate a Polymarket copy trading research system.

The system should:

1. Pull the Polymarket or Bullpen leaderboard.
2. Scan the top 500 wallets.
3. Analyze the last 30 days of wallet activity.
4. Score each wallet by ROI, consistency, and copyability.
5. Penalize wallets where profit came from one lucky trade.
6. Rank wallets globally and by category.
7. Skip wallets or markets that are too illiquid to copy.
8. Track selected wallets.
9. Detect new trades.
10. Score whether each new trade is worth copying.
11. Paper trade copy candidates using simulated position sizes between \$5 and \$20.
12. Update paper PnL every hour.
13. Track final outcomes when markets resolve.
14. Compare the bot-filtered strategy against blindly copying leaderboard wallets.
15. Track missed winners and avoided losers.
16. Automatically update rules based on performance.
17. Record every rule change with version history.
18. Send an end-of-day report through Hermes.
19. Show performance in a clean Max HQ dashboard.

The app should have two layers:

Layer 1: Hermes Agent operator

Hermes should run the operational loop:

- scheduled leaderboard scans
- wallet profile updates
- new trade monitoring
- paper trade tracking
- hourly PnL updates
- outcome reviews
- automatic rule updates
- end-of-day reports
- important Telegram alerts only
- weekly performance summaries

Layer 2: Vercel dashboard

The dashboard should be deployable on Vercel and designed to fit inside Max HQ.

The dashboard should look polished, focused, and not overwhelming.

It should answer three questions immediately:

1. Are we profitable on paper?
2. Which wallets are worth copying?
3. What did the bot learn today?

Tech stack:

- TypeScript

- Next.js
- React
- Tailwind
- SQLite locally
- Prisma or Drizzle
- Vercel-ready architecture
- Polymarket public APIs where possible
- Adapter layer for leaderboard, market data, trades, prices, and outcomes
- No paid services required for version one

Create these pages:

1. Overview

Show:

- total paper PnL
- win rate
- open paper positions
- active tracked wallets
- copy candidates today
- end-of-day report status
- latest rule changes
- chart of paper PnL over time

2. Wallet Rankings

Show:

- top 500 leaderboard scan
- wallet address
- wallet label
- global rank
- ROI
- consistency score
- copyability score
- one-hit-wonder penalty
- best category
- status: track, watch, ignore
- reason for status

3. Wallet Profile

For each wallet show:

- ROI over last 30 days
- number of trades
- win rate on resolved markets
- average trade size
- category strengths
- liquidity profile
- average entry timing
- whether it is copyable
- whether it is too early, too late, too illiquid, or category-specific
- recent trades
- paper performance if copied

4. Trade Signals

Show:

- new wallet trades
- market question
- wallet entry price
- current price
- price movement since wallet entry
- spread
- liquidity
- time to resolution
- decision: paper_copy, watchlist, skip
- score
- reason
- risk

5. Paper Trades

Show:

- simulated trades
- simulated position size between \$5 and \$20
- entry price
- current price

- hourly PnL
- final PnL if resolved
- current status
- reason for entering
- linked wallet and market

6. Decision Journal

Show:

- every decision
- copy, watchlist, or skip
- score breakdown
- reasons
- risks
- whether the decision was later judged good or bad
- what the bot learned

7. Performance

Show:

- PnL chart
- win rate chart
- hourly PnL updates
- category performance
- wallet performance
- bot-filtered strategy vs blind leaderboard copy
- missed winners
- avoided losers

8. Rules

Show:

- active rule version
- current scoring thresholds
- automatic rule changes
- why each rule changed
- before and after values
- evidence used
- rule version history

9. Reports

Show:

- end-of-day reports
- weekly reports
- best wallets today
- worst wallets today
- important rule updates
- performance summary
- what Hermes changed and why

Database models:

LeaderboardScan:

- id
- source
- scannedAt
- walletCount
- lookbackDays
- rawSummaryJson

WalletProfile:

- id
- address
- label
- sourceRank
- status: track, watch, ignore
- roi30d
- consistencyScore
- copyabilityScore
- oneHitWonderPenalty
- globalScore
- bestCategory
- categoryStrengthsJson
- averageTradeSize

- tradeCount30d
- resolvedTradeCount30d
- winRate30d
- averageLiquidity
- averageSpread
- averageEntryTiming
- copyabilityNotes
- riskNotes
- lastScannedAt
- createdAt
- updatedAt

ObservedTrade:

- id
- walletAddress
- marketId
- conditionId
- marketQuestion
- marketCategory
- outcome
- side
- walletEntryPrice
- detectedPrice
- size
- timestamp
- rawTradeJson
- createdAt

MarketSnapshot:

- id
- marketId
- conditionId
- question
- category
- yesPrice
- noPrice
- bestBid
- bestAsk
- spread
- liquidity
- volume
- timeToResolution
- collectedAt
- rawMarketJson

DecisionJournal:

- id
- observedTradeId
- walletAddress
- marketId
- decision: paper_copy, watchlist, skip
- copyScore
- confidence
- reasonsJson
- risksJson
- walletQualityScore
- roiScore
- consistencyScore
- copyabilityScore
- categoryFitScore
- entryTimingScore
- spreadScore
- liquidityScore
- thesisScore
- simulatedPositionSize
- createdAt

PaperTrade:

- id
- decisionJournalId
- walletAddress
- marketId
- outcome
- side

- entryPrice
- currentPrice
- simulatedPositionSize
- unrealizedPnl
- realizedPnl
- status: open, closed, resolved
- openedAt
- closedAt
- resolvedAt

PnlSnapshot:

- id
- paperTradeId
- price
- pnl
- collectedAt

OutcomeReview:

- id
- decisionJournalId
- paperTradeId
- reviewTime
- priceAfter1h
- priceAfter6h
- priceAfter24h
- finalOutcome
- simulatedPnl
- wasDecisionGood
- lessonsJson
- createdAt

RuleSet:

- id
- version
- active
- rulesJson
- createdAt
- updatedAt

RuleChange:

- id
- oldRuleSetId
- newRuleSetId
- changedBy: hermes
- reason
- evidenceSummary
- beforeJson
- afterJson
- createdAt

DailyReport:

- id
- date
- paperPnl
- winRate
- openPositions
- newSignals
- copiedSignals
- watchedSignals
- skippedSignals
- bestWalletsJson
- worstWalletsJson
- ruleChangesJson
- summary
- sentToTelegram
- createdAt

Scoring requirements:

Wallet scoring:

Score wallets by:

- ROI
- consistency
- copyability
- category edge
- liquidity quality
- entry timing
- trade frequency
- resolved trade performance
- one-hit-wonder penalty

A wallet should be penalized if:

- most profit came from one trade
- trades are too illiquid
- price moves too far after entry
- edge only exists in one old market
- there are not enough resolved trades
- spread is usually too wide
- the bot cannot realistically follow entries

Trade scoring:

Score each new wallet trade by:

- wallet global score
- wallet category score
- current market price
- wallet entry price
- price movement since wallet entry
- spread
- liquidity
- time to resolution
- current rule thresholds
- thesis clarity

Decision labels:

- paper_copy: strong enough to simulate a copy trade
- watchlist: interesting but not clean enough
- skip: too late, too illiquid, weak wallet, bad category fit, or poor setup

Paper trading:

- For every paper_copy decision, create a PaperTrade.
- Simulated bet size should be between \$5 and \$20.
- Higher confidence can use larger simulated size.
- Update PnL every hour.
- Close or resolve paper trades when the market resolves or when rules say the trade should be exited.
- Track paper PnL over time.

Benchmarks:

Compare:

1. Bot-filtered paper trades
2. Blind copy of leaderboard wallets
3. Watchlist trades
4. Skipped trades

Track:

- missed winners
- avoided losers
- bad copies
- good skips
- late entries avoided
- spread losses avoided

Self-improvement:

The system should automatically update rules.

It should not ask for approval before changing paper-trading rules.

But every rule change must be logged and explained.

For each rule change, store:

- what changed
- why it changed
- evidence used

- before value
- after value
- expected improvement
- timestamp
- new rule version

Examples:

- lower max spread threshold if spread-heavy trades underperform
- raise minimum liquidity if low-liquidity trades perform poorly
- downgrade wallets with poor recent paper performance
- mark wallets as category-specific
- reduce allowed price movement if late entries lose
- increase consistency weighting if high-ROI wallets are too volatile

Reports:

Create an end-of-day report that Hermes can send to Telegram.

The report should include:

- paper PnL today
- total paper PnL
- win rate
- best paper trade
- worst paper trade
- best wallet today
- worst wallet today
- rule changes made
- top lesson learned
- whether the bot-filtered strategy beat blind copy today
- what to watch tomorrow

Alerts:

Keep Telegram alerts minimal.

Minimum one end-of-day report per day.

Only send additional alerts for genuinely important events, like:

- a very high-confidence paper trade
- a major rule change
- a wallet being upgraded or downgraded significantly
- a performance drawdown warning

Testing:

Create tests for:

- wallet scoring
- one-hit-wonder penalty
- copyability score
- trade scoring
- paper trade creation
- hourly PnL updates
- rule versioning
- automatic rule changes
- benchmark comparison
- read-only safety
- no real trade execution

Commands:

- npm run dev
- npm run db:migrate
- npm run seed
- npm run scan:leaderboard
- npm run scan:wallets
- npm run monitor:trades
- npm run score:trades
- npm run paper:update-pnl
- npm run review:outcomes
- npm run update:rules
- npm run report:daily
- npm run test

README:

Include:

- what the bot does
- what the bot does not do

- safety explanation
- setup instructions
- environment variables
- how to run locally
- how to deploy to Vercel
- how to add it to Max HQ
- how Hermes should operate it
- how the leaderboard scan works
- how wallet scoring works
- how paper trading works
- how self-improvement works
- how to interpret the dashboard

Also create SAFETY.md:

Explain:

- why version one is paper trading only
- why real execution is disabled
- how autonomy could be added later
- risks of stale data
- risks of low liquidity
- risks of wide spreads
- risks of copy trading
- why leaderboard wallets can be misleading
- why private keys should never be stored in the app

Implementation order:

1. Create project structure.
2. Create database schema.
3. Create Polymarket and leaderboard adapters.
4. Build leaderboard scanner.
5. Build wallet profiler.
6. Build trade monitor.
7. Build trade scorer.
8. Build paper trading engine.
9. Build hourly PnL updater.
10. Build outcome reviewer.
11. Build automatic rule updater.
12. Build dashboard.
13. Build daily report generator.
14. Add Hermes operator prompts and cron examples.
15. Add tests.
16. Run tests.
17. Show me what works and what still needs manual setup.

After building, run the tests and report:

- commands run
- tests passed
- tests failed
- API blockers
- manual setup needed
- how to run the dashboard locally
- how to deploy it to Vercel